Secure File Storage (AES-256)

Elevate Labs — Cyber Security Internship | 2025 | Nithish Nidumolu

1. Introduction

The Secure File Storage project is a Python-based security tool developed as part of the Elevate Labs Cyber Security Internship (2025). It is designed to encrypt and decrypt files using the AES-256-GCM algorithm — one of the strongest symmetric encryption standards available. The tool targets sensitive files of any type and protects them with a password-derived 256-bit key, ensuring that only authorised users with the correct passphrase can access the original content. A Flask-based web interface hosted on Replit allows users to encrypt and decrypt files directly from their browser.

2. Abstract

The tool accepts any file type and encrypts it using AES-256-GCM with a password-derived key and a random 16-byte Initialisation Vector (IV). The encrypted output is saved as a .enc file and can be decrypted by any user who possesses the correct password. A Flask web interface handles file upload, encryption, decryption, and download. Every operation is logged to a structured activity log (file_storage_log.txt) recording the file name, operation type, timestamp, and status. The GCM authentication tag ensures that tampered or incorrectly decrypted files are rejected with a clear error message.

3. Tools & Technologies

- Python 3.8+ — Core programming language
- Flask — Lightweight web framework for the UI and routing
- PyCryptodome — AES-256-GCM encryption and decryption engine
- cryptography — Alternative AES implementation with hazmat primitives
- hashlib / os.urandom — SHA-256 key derivation and random IV generation
- Jinja2 — HTML templating for dynamic result rendering
- Replit — Cloud deployment platform

4. Steps Involved in Building the Project

- Project Setup: Created the folder structure with app.py, crypto.py, templates/ (index.html, result.html), and static/ (style.css). Set up a virtual environment and installed Flask, PyCryptodome, and cryptography.
- Key Derivation: Implemented derive_key(password) using SHA-256 to produce a deterministic 32-byte AES key. The key is never stored on disk — it exists only in memory during the operation.
- Encryption Engine: Built encrypt_file() to generate a random 16-byte IV and encrypt file bytes using AES-256-GCM. Output format: IV + GCM tag + ciphertext saved as .enc.
- Decryption Engine: Built decrypt_file() to extract the IV and tag, verify the GCM authentication tag, and return plaintext bytes. Incorrect passwords trigger an Authentication tag mismatch error.
- Flask Routes: Defined / (home), /encrypt (POST), and /decrypt (POST) routes. Each calls the crypto function and returns the processed file via Flask send_file().
- Activity Logging: Implemented append_log() to write structured entries to file_storage_log.txt after every operation, including timestamp, file name, and status.
- Frontend: Built index.html with a file upload widget and password field, and result.html with colour-coded SUCCESS/FAILED status badges using Jinja2 templates.
- Deployment: Configured the Flask server for Replit (host=0.0.0.0, port=5000) and deployed publicly.

5. Conclusion

The Secure File Storage tool successfully demonstrates AES-256-GCM encryption and decryption in a production-ready, browser-accessible Python application. The tool ensures confidentiality through strong symmetric encryption, data integrity through GCM authentication tags, and non-repudiation through structured activity logs. The modular Flask architecture separates the web layer, encryption engine, and logging output, making the tool scalabl

Features & Functionality

Core Features	
AES-256-GCM Encryption	Encrypts any file type using the AES-256-GCM authenticated encryption scheme, providing both confidentiality and data integrity in a single operation.
Random IV Generation	A unique 16-byte Initialisation Vector (IV) is generated for every encryption — ensuring that encrypting the same file twice always produces completely different ciphertext.
GCM Tag Verification	Before any decryption, the AES-GCM authentication tag is verified. A tampered file or wrong password is immediately rejected with an 'Authentication tag mismatch' error — no partial decryption occurs.
Password-Based Key Derivation	Uses SHA-256 to derive a deterministic 256-bit symmetric key from the user's password. The key never touches disk — it lives only in memory during the operation.
Browser File Upload & Download	Clean Flask web interface where users upload any file, provide a password, and instantly download the .enc encrypted output or the restored plaintext file.
Activity Log Generation	Every encrypt and decrypt operation automatically appends a structured entry to file_storage_log.txt — including filename, operation type (ENCRYPT/DECRYPT), timestamp, file size, and status (SUCCESS/FAILED).
Colour-Coded Results	Results page renders a green SUCCESS badge on successful operations and a red FAILED badge with a descriptive error message on failures, making status immediately clear.
Cross-Platform Support	Runs on Windows 10/11, macOS, and Linux. Deployable on Replit, Heroku, or any cloud platform with minimal configuration changes.

6. Scenario-Based Case Study

Scenario: Riya, a junior data analyst at a healthcare company, needs to securely send a patient records CSV to a colleague over email. Her IT security team requires the file to be encrypted using AES-256 before transmission to comply with data protection policies.

Step 1 — Accessing the Tool: Riya opens her browser and navigates to https://aes256-storage--nithishnidumolu.replit.app. She sees the clean interface with a file upload widget, a password field, and the Encrypt File button.

Step 2 — Encrypting the File: She selects patient_records.csv, enters the password P@ssw0rd#Secure2025, and clicks Encrypt File. The backend derives the AES-256 key via SHA-256, generates a random 16-byte IV, and encrypts the file using AES-256-GCM. A download link for patient_records.csv.enc appears on screen.

Step 3 — Downloading & Sending: Riya downloads patient_records.csv.enc and attaches it to the email. Even if intercepted in transit, the contents are completely unreadable without the password.

Step 4 — Decryption by Recipient: Her colleague uploads patient_records.csv.enc, enters the shared password, and clicks Decrypt File. The backend verifies the GCM authentication tag and restores the original patient_records.csv — byte-for-byte identical to the original.

Step 5 — Reviewing the Activity Log: Riya opens file_storage_log.txt and shares it with her IT compliance team as proof of compliant data handling. The log clearly shows ENCRYPT and DECRYPT operations, both with SUCCESS status.